

FINAL 10/12/2025

1- Un equipo de desarrollo de tres integrantes se encuentra trabajando en un proyecto ágil, utilizando la técnica de poker planning y utilizando la siguiente secuencia de Fibonacci (1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987) para asignar pesos a las tareas. Al comenzar la estimación de tareas se encuentran con el siguiente conflicto: Uno considera que una tarea debe recibir el peso de 144, sin embargo, los otros dos uno cree que debe ser de 233 y el otro 89. En la siguiente les pasa lo mismo, pero con los valores 3, 5 y 8. Eso les pasa en casi todas las tareas, y les cuesta llegar a un consenso por lo que pasan mucho tiempo analizando y discutiendo las estimaciones. ¿Qué les recomendarías?

Les recomendaría, en primer lugar, **reducir la escala de estimación**, utilizando valores más acotados de la secuencia de Fibonacci (por ejemplo hasta 13 o 21), ya que el Planning Poker busca estimaciones relativas y no una precisión excesiva.

Ante diferencias marcadas en las votaciones, es importante **verificar que todos los integrantes comprendan correctamente la tarea**, ya que la falta de entendimiento o la percepción de distintos riesgos suele generar dispersión en las estimaciones.

Luego, se debería **comparar la tarea con otras previamente estimadas** sobre las cuales el equipo haya alcanzado consenso, para usarla como referencia. Finalmente, tras discutir los motivos de las estimaciones más altas y más bajas, se realiza una nueva votación hasta llegar a un consenso. Igualmente, si todos están de acuerdo se podría definir por el valor 144 que es el intermedio entre los 3 obtenidos.

2- ¿Qué es un **Code Smell**? Nombre 2 y detalle y explique cada uno.

Un *code smell* es una señal en el código que indica un posible problema de diseño. No implica necesariamente que el programa funcione mal o produzca resultados incorrectos, sino que sugiere que el código es difícil de mantener, extender o comprender, y que podría mejorarse mediante refactorización.

Un ejemplo de *code smell* es el **código duplicado**, que ocurre cuando el mismo bloque de código aparece repetido en varias partes del sistema. Esto dificulta el mantenimiento, ya que ante un cambio es necesario modificar todas las copias, aumentando el riesgo de errores y haciendo el código más largo e innecesariamente repetitivo.

Otro *code smell* son las **clases o métodos demasiado largos**, que concentran demasiadas responsabilidades. Esto viola el principio de responsabilidad única (SRP) y dificulta la extensión del código sin modificarlo, afectando también el principio abierto/cerrado (O/C). Como consecuencia, el código se vuelve más complejo, menos legible y más difícil de mantener. Además, es muy difícil poder reutilizar, por ejemplo, métodos largos ya que suelen ser para casos muy específicos.

3- Cuál/es es/son la/s diferencia/s entre los patrones **Strategy** y **State**. Focalice en las diferencias y similitudes, no en contar cada uno por separado. Desarróllalo y haga un ejemplo concreto con diagramas de un caso donde podría aplicar para cada uno.

Los patrones **Strategy** y **State** son estructuralmente muy similares: ambos se basan en **composición y polimorfismo**, utilizando una interfaz común y múltiples implementaciones concretas para encapsular comportamientos y evitar grandes estructuras condicionales (**if/switch**), lo que ayuda a prevenir *code smells*.

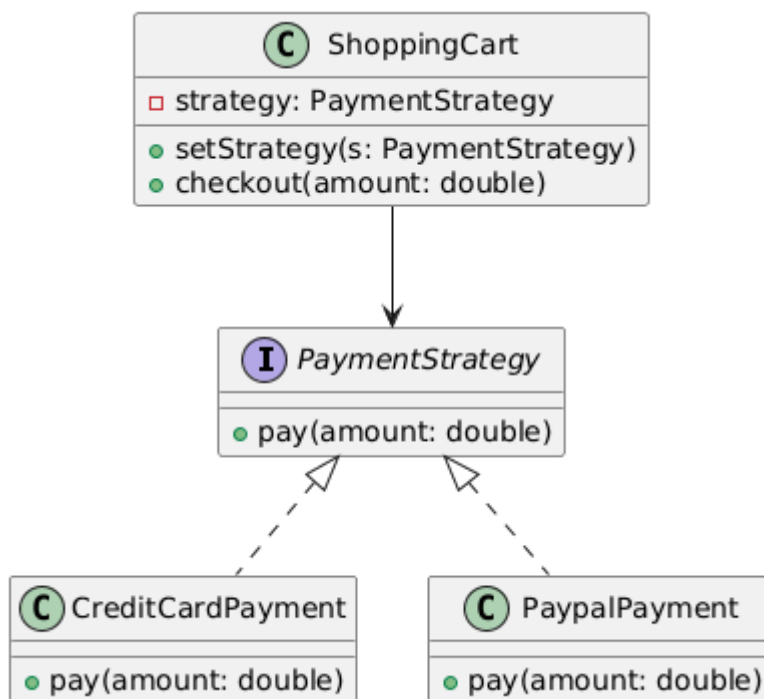
La principal diferencia entre ambos patrones está en **la intención y el control del cambio de comportamiento**.

En **Strategy**, el objetivo es **definir una familia de algoritmos intercambiables**. El contexto delega el comportamiento a una estrategia concreta, pero **no decide cuándo cambiarla**: la estrategia suele ser seleccionada desde el exterior (por ejemplo, por configuración o por el cliente). Las estrategias **no se conocen entre sí** y no controlan el flujo del contexto.

En cambio, **State** modela **el comportamiento de un objeto según su estado interno**. El contexto cambia su comportamiento cuando cambia su estado, y **los estados pueden conocerse entre sí y provocar transiciones**. Es decir, el cambio de comportamiento ocurre de manera automática y controlada internamente por los estados.

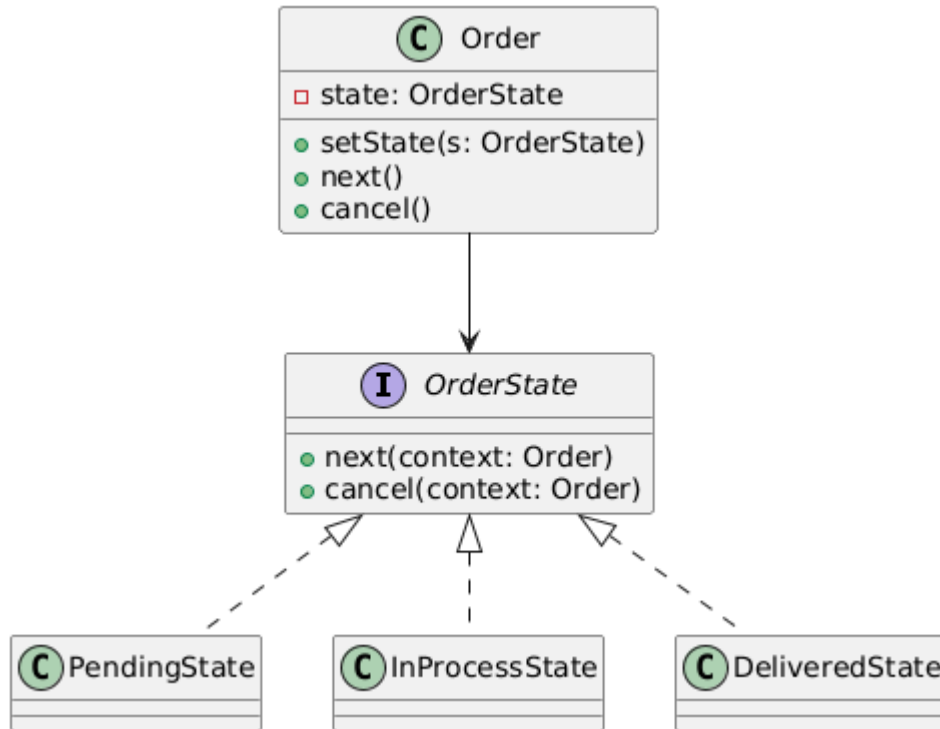
En resumen, **Strategy cambia el “cómo” se hace algo**, mientras que **State cambia el “qué se puede hacer” según la situación del objeto**, aunque ambos compartan una estructura similar.

Ejemplo Strategy



Idea clave: el **ShoppingCart** no sabe *cuándo* cambiar la estrategia; simplemente usa la que le asignan.

Ejemplo state



Idea clave: cada estado decide a qué estado se pasa y **controla la transición**.

Aunque Strategy y State comparten una estructura similar, se diferencian en su intención: Strategy encapsula algoritmos intercambiables elegidos externamente, mientras que State encapsula comportamientos dependientes del estado, donde las transiciones son controladas internamente por los propios estados.

4-

1. Analice el siguiente código y comente si encuentra alguna mala práctica en su diseño o implementación.
2. Se desea agregar soporte para archivos CSV. Agregue dicha funcionalidad respetando las buenas prácticas, y usando principios o patrones y técnicas vistas en la materia.

```
class JSONDataProcessor {

    public void process(String fileName) {

        System.out.println("Loading data...");
```

```
String content = this.readFile(fileName);

JSONObject jsonContent = this.parseJson(content);

if (this.validateJson(jsonContent)) {

    Object extracted = this.extractFromJson(jsonContent);

    this.processContent(extracted);

} else {

    throw new IllegalArgumentException("Contenido inválido");

}

this.saveToJson(jsonContent, fileName);

this.saveMetadata(jsonContent);

System.out.println("Processing complete.");

}

// Métodos auxiliares (simulados con printins)

private String readFile(String fileName) {

    return ""; // simulado

}

private JSONObject parseJson(String content) {

    System.out.println("Parsing JSON: " + content);

    return new JSONObject();

}

private boolean validateJson(JSONObject json) {

    System.out.println("Validating JSON structure");

    return true;

}
```

```
private Object extractFromJson(JSONObject json) {

    System.out.println("Extracting content from JSON");

    return new Object();

}

private void processContent(Object content) {

    System.out.println("Processing content");

}

private void saveToJson(JSONObject json, String fileName) {

    System.out.println("Saving JSON");

}

private void saveMetadata(Object content) {

    System.out.println("Saving Metadata");

}

}

class XMLDataProcessor {

    public void process(String fileName) {

        System.out.println("Loading data...");

        String content = this.readFile(fileName);

        XMLObject xmlContent = this.parseXML(content);

        if (this.validateXML(xmlContent)) {

            Object extracted = this.extractFromXml(xmlContent);

            this.processContent(extracted);

        } else {

            throw new IllegalArgumentException("Contenido inválido");

        }

    }

}
```

```
}

    this.saveToXML(xmlContent, fileName);

    this.saveMetadata(xmlContent);

    System.out.println("Processing complete.");
}

// Métodos auxiliares simulados

private String readFile(String fileName) {

    return ""; // simulado

}

private XMLObject parseXML(String content) {

    System.out.println("Parsing XML: " + content);

    return new XMLObject();

}

private boolean validateXML(XMLObject xml) {

    System.out.println("Validating XML structure");

    return true;

}

private Object extractFromXml(XMLObject xml) {

    System.out.println("Extracting content from XML");

    return new Object();

}

private void processContent(Object content) {

    System.out.println("Processing content");

}
```

```

private void saveToXML(XMLObject xml, String fileName) {

    System.out.println("Saving XML");

}

private void saveMetadata(Object content) {

    System.out.println("Saving Metadata");

}

// Clase ficticia para simular XML

class XMLObject {

    // Simulación de una estructura XML parseada

}

}

```

Salida esperada:

- **JSON**

```

Loading data...

Parsing JSON: {"name": "John"}

Validating JSON structure

Extracting content from JSON

Processing content

Saving JSON

Saving Metadata

Processing complete.

```

- **XML**

```

Loading data...

Parsing XML: <name>John</name>

Validating XML structure

```

```
Extracting content from XML
Processing content
Saving XML
Saving Metadata
Processing complete.
```

- **CSV (a implementar)**

```
Loading data...
Parsing CSV: name,John
Validating CSV structure
Extracting content from CSV
Processing content
Saving CSV
Saving Metadata
Processing complete.
```

a- En el diseño presentado se observan varias malas prácticas: **Código duplicado**:

JSONDataProcessor y **XMLDataProcessor** tienen prácticamente el mismo flujo de procesamiento, cambiando solo el formato específico. **Violación del principio Open/Closed (OCP)**: para agregar soporte a un nuevo formato (por ejemplo CSV) sería necesario crear otra clase copiando y modificando código existente. **Violación del principio de Responsabilidad Única (SRP)**: cada clase se encarga de múltiples responsabilidades (lectura, parseo, validación, extracción, procesamiento y guardado). **Alto acoplamiento al formato**: el flujo general está mezclado con los detalles de cada tipo de archivo. **Difícil extensibilidad**: agregar nuevos formatos incrementa el código duplicado y el riesgo de errores.

b- Idea general:

- **Strategy** se utiliza para encapsular el comportamiento de procesamiento según el formato del archivo (JSON, XML, CSV).
- **Interpreter** se usa para interpretar la estructura del contenido de cada formato (por ejemplo, cómo leer campos y valores desde JSON, XML o CSV).

De esta manera: El sistema queda **abierto a nuevos formatos** (OCP). Se elimina la duplicación de código. Cada formato define su propia forma de interpretar los datos. El uso combinado de Strategy e Interpreter permite desacoplar el flujo de procesamiento del formato de los datos, facilitando la incorporación de nuevos tipos de archivos como CSV sin modificar el código existente y respetando los principios de diseño vistos en la materia.

```
// Strategy
interface DataStrategy {
    Object parse(String content);
    boolean validate(Object data);
    void save(Object data);
}
```

```

}
// Interpreter
interface Expression {
    Object interpret(String content);
}
// Interpreter concreto CSV
class CSVExpression implements Expression {
    public Object interpret(String content) {
        System.out.println("Parsing CSV: " + content);
        return new Object();
    }
}
// Strategy concreta CSV
class CSVStrategy implements DataStrategy {

    private Expression interpreter = new CSVExpression();

    public Object parse(String content) {
        return interpreter.interpret(content);
    }

    public boolean validate(Object data) {
        System.out.println("Validating CSV structure");
        return true;
    }

    public void save(Object data) {
        System.out.println("Saving CSV");
    }
}
// Contexto
class DataProcessor {

    private DataStrategy strategy;

    public DataProcessor(DataStrategy strategy) {
        this.strategy = strategy;
    }

    public void process(String content) {
        Object data = strategy.parse(content);
        if (strategy.validate(data)) {
            System.out.println("Processing content");
        }
    }
}

```

```
        strategy.save(data);  
        System.out.println("Saving Metadata");  
    }  
}  
}
```

Pregunta adicional: Le encargan repartir 2 limones entre 3 personas interesadas en los mismos. ¿Qué haría? Justifique su respuesta.

Para poder ofrecer una buena solución, primero es fundamental comprender qué necesita realmente cada usuario y cuál es el problema que quiere resolver. No todas las personas buscan lo mismo, aun cuando el recurso sea el mismo. Por ejemplo, alguien puede necesitar solo la cáscara del limón para una receta, otra persona puede estar interesada únicamente en el jugo para una ensalada, y otra quizás quiera aprovechar las semillas para cultivar una planta. Por eso, antes de proponer o diseñar cualquier solución, es clave hacer las preguntas correctas y no dar por sentado que todos tienen los mismos objetivos. De esta manera, se logra brindar un valor real y adaptado a cada necesidad particular.